

Proyecto Corte 1

Materia: Frameworks y Herramientas para Big Data

Grupo: 2

Integrantes:

- Nicolás Santiago Cuarán Sotelo
- Sebastián Belalcázar Mosquera
- Bryan Fernando Burbano Carvajal
- Michel Dahiana Burgos Santos
- Juan David Daza Rivera

Fecha: 08/09/2026

Repositorio: <https://github.com/SEBASBELMOS/nyc-taxi-lakehouse>

1. Resumen

Se construyó un data stack completo sobre Docker Compose: ingesta de un Parquet público de NYC Yellow Taxi (enero 2025), aterrizaje en un data lake (MinIO), conversión a tabla Apache Iceberg catalogada en Nessie (vía Iceberg REST) y distribución final hacia Azure ADLS y ClickHouse. Toda la ingesta y carga se realiza con `dlt`; la lectura de Parquet se hace con PyArrow en lotes.

Resultado de la validación crítica: la tabla final en ClickHouse contiene exactamente **3.475.226 filas**, verificado de forma independiente en tres capas: RAW (MinIO), tabla Iceberg (vía Nessie) y ClickHouse. La verificación integral del stack cierra en **12/12 checks**, incluida la confirmación de los archivos cargados en Azure.

2. Entregables del enunciado

Entregable	Dónde está
4 scripts de ingesta y cargue de datos	<code>scripts/01..04_*.py</code> , ejecutados como notebooks en Jupyter (<code>notebooks/01..04_*.ipynb</code> , con outputs)
1 archivo <code>docker-compose.yml</code>	raíz del proyecto (4 servicios: <code>minio</code> , <code>nessie</code> , <code>clickhouse</code> , <code>jupyter</code>)
1 archivo con los secrets de <code>dlt</code>	<code>dlt/secrets.toml</code> (incluido en la carpeta de entrega; excluido del repositorio público)
Pantallazos solicitados	9 capturas en <code>evidencias/</code> , incluidas en la sección 7
Servicios corriendo en Docker	Figura 1
Dos buckets de MinIO con sus Parquet	Figuras 2, 3 y 4
Catálogo de Nessie con el namespace	Figura 5
Consulta con cantidad de registros (3475226)	Figura 6
Consulta con tablas de metadatos en ClickHouse	Figura 7
Consulta del archivo cargado en Azure	Figura 8 (verificación programática con <code>adlfs</code>)

3. Arquitectura

Función	Tecnología	Servicio Docker
Data Lake / Object Storage	MinIO	<code>minio</code>
Catálogo	Nessie (Iceberg REST)	<code>nessie</code>
Data Warehouse	ClickHouse	<code>clickhouse</code>
Entorno interactivo	Jupyter	<code>jupyter</code>
File format	Apache Parquet	—
Table format	Apache Iceberg	—
Ingesta / carga	<code>dlt</code>	—
Lectura de Parquet	PyArrow	—



4. Configuración y decisiones técnicas

- **Secretos:** las credenciales de Azure, MinIO y ClickHouse viven en `dlt/secrets.toml` (excluido de control de versiones y montado como volumen de solo lectura en el contenedor de Jupyter). Ningún script ni notebook contiene credenciales, y ninguna aparece en los outputs guardados.
- **Entorno:** `.env` con los parámetros no secretos; `GROUP_FOLDER=GRUPO_2`.
- **Nessie como catálogo Iceberg REST:** Pylceberg no tiene un tipo de catálogo `nessie`; Nessie expone el protocolo Iceberg REST en `/iceberg` y el cliente se conecta con `RestCatalog`.
- **Catálogo persistente:** version store de Nessie en RocksDB sobre volumen (el default `IN_MEMORY` pierde namespace y tabla al reiniciar).
- **Memoria controlada:** la fuente (~3,5 M de filas) nunca se carga entera: `ParquetFile.iter_batches(batch_size=250.000)` y entrega de lotes PyArrow.
- **Puertos:** ClickHouse nativo se publica en el 9002 del host porque MinIO ocupa el 9000; dentro de la red Docker ClickHouse sigue en su 9000.
- **Nombres legibles:** `dataset_table_separator = "_"` en `dlt/config.toml`; la tabla final es `nyc_taxi_yellow_tripdata_2025_01` (el default de dlt uniría dataset y tabla con

tres guiones bajos).

- **Dependencias declaradas:** todas en `requirements.txt` e instaladas en la imagen (`Dockerfile`); no hay `pip install` dentro de los notebooks.

5. Ejecución de los pipelines

Los cuatro pipelines se ejecutaron **como notebooks dentro de Jupyter** (`notebooks/01..04`, con sus outputs guardados como evidencia). Las versiones `.py` equivalentes están en `scripts/` y pueden ejecutarse con `docker compose exec jupyter python scripts/<script>.py`; ambos formatos comparten la misma configuración (`scripts/config.py`).

5.1. Pipeline 01 — HTTP → MinIO RAW

- Fuente: `https://d37ci6vzurychx.cloudfront.net/trip-data/yellow_tripdata_2025-01.parquet`
- Destino: `s3://nyc-taxi-raw/raw/yellow_tripdata/`
- Resultado: 1 archivo Parquet de 72,9 MB (`1788272855.525187.7eef0a56d0.parquet`).

```
Rows landed in MinIO: 3,475,226
OK - RAW bucket matches the expected row count.
```

5.2. Pipeline 02 — MinIO RAW → Iceberg + Nessie

- Tabla creada: `nyc_taxi.yellow_tripdata_2025_01` (20 columnas)
- Location: `s3://nyc-taxi-iceberg/nyc_taxi/yellow_tripdata_2025_01_25fa2203-bb5d-49f6-8162-bc570ef08446`
- Snapshots: 1 · Data files: 14 · Metadata files: 43
- Carga por lotes de 250.000 filas (`iter_batches` + `append`); antes de escribir se eliminan las columnas internas de dlt (`_dlt_load_id`, `_dlt_id`) para que la tabla Iceberg sea copia fiel del origen.

```
[OK] Iceberg row count matches - 3,475,226 (expected 3,475,226)
```

5.3. Pipeline 03 — Iceberg → Azure ADLS

- Origen: los data files se resuelven **preguntando al catálogo de Nessie** (`table.location()`), sin rutas hardcodedas.
- Destino: `abfss://clase-4-dlt@fhbd.dfs.core.windows.net/GRUP0_2/nyc_taxi/`
- Resultado: data files Parquet (~73 MB) más la metadata de dlt, confirmados mediante verificación programática con `adlfs` (listado del contenedor con las mismas credenciales, sin depender del portal de Azure).

5.4. Pipeline 04 — Iceberg → ClickHouse

- Tabla final: `default.nyc_taxi_yellow_tripdata_2025_01`
- Tablas de metadatos de dlt creadas: `nyc_taxi__dlt_loads`, `nyc_taxi__dlt_pipeline_state`, `nyc_taxi__dlt_version`, `nyc_taxi_dlt_sentinel_table`.
- Carga completa en 3,02 s; `write_disposition="replace"` garantiza el conteo exacto en cada re-ejecución.

```
SELECT count(*) FROM default.nyc_taxi_yellow_tripdata_2025_01;
-> 3,475,226
[OK] ClickHouse row count matches - 3,475,226 (expected 3,475,226)
```

6. Verificación integral

`scripts/verificar.py` (también ejecutado como `notebooks/verificar.ipynb`) recorre todo el stack en una sola corrida: **12/12 checks pasados**.

```
[OK ] bucket 'nyc-taxi-raw' exists
[OK ] bucket 'nyc-taxi-iceberg' exists
[OK ] RAW bucket has Parquet files - 1 file(s)
[OK ] Iceberg bucket has data files - 42 file(s)
[OK ] Iceberg bucket has metadata files - 129 file(s)
[OK ] namespace 'nyc_taxi' exists
[OK ] table 'nyc_taxi.yellow_tripdata_2025_01' registered
[OK ] Iceberg row count matches - 3,475,226 (expected 3,475,226)
[OK ] table 'nyc_taxi_yellow_tripdata_2025_01' exists
[OK ] dlt metadata tables present
[OK ] ClickHouse row count matches - 3,475,226 (expected 3,475,226)
[OK ] files present in Azure - 5 file(s)
```

Los conteos de archivos de este reporte son acumulados del bucket, no de la tabla vigente: el pipeline 02 recrea la tabla en cada corrida pero los data files de las corridas anteriores permanecen en el bucket. La tabla activa tiene los 14 data files y 43 archivos de metadata reportados en la sección 5.2; el conteo de filas, que es la validación crítica, se toma de la tabla y da exacto.

7. Evidencias

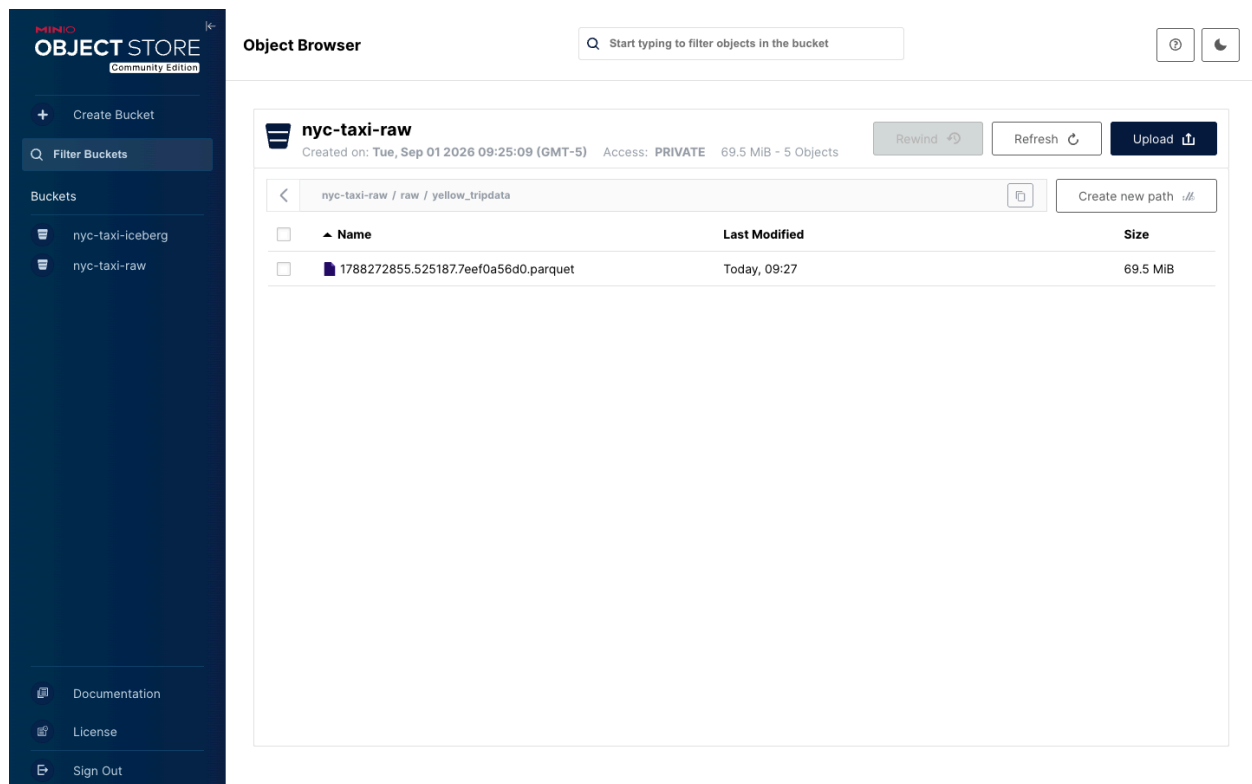
Las nueve capturas siguientes documentan cada punto solicitado por el enunciado. Los archivos originales están en la carpeta `evidencias/` del proyecto.

Figura 1 — Servicios corriendo en Docker



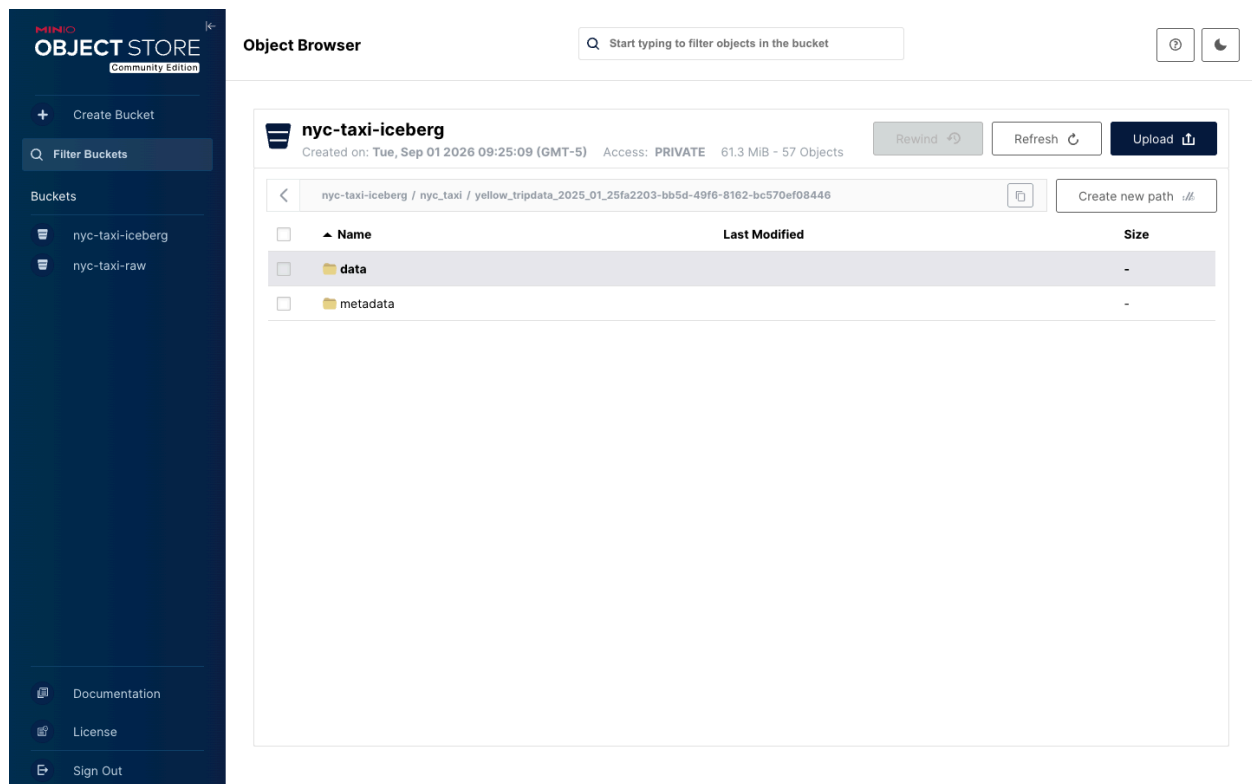
Los cuatro servicios del stack activos: `minio`, `nessie`, `clickhouse` y `jupyter`. Los contenedores `volume-init` y `minio-init` aparecen finalizados (`Exited 0`) porque son tareas de inicialización: ceder la propiedad de los volúmenes y crear los buckets.

Figura 2 — Bucket RAW en MinIO



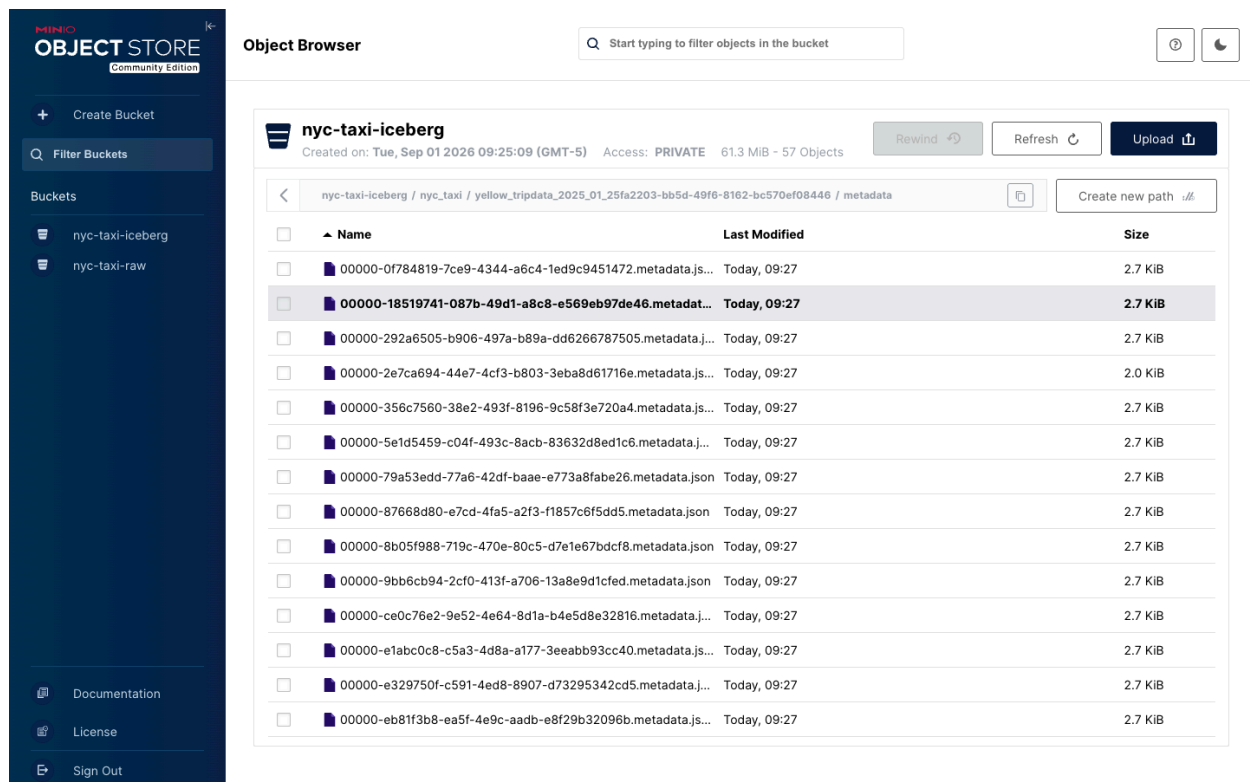
Bucket `nyc-taxi-raw` con el archivo Parquet aterrizado por el pipeline 01 (72,9 MB, 3.475.226 filas).

Figura 3 — Bucket Iceberg en MinIO



Bucket `nyc-taxi-iceberg` con la estructura de la tabla Apache Iceberg: carpeta `data/` con los archivos Parquet y `metadata/` con los manifiestos y snapshots.

Figura 4 — Metadata de la tabla Iceberg



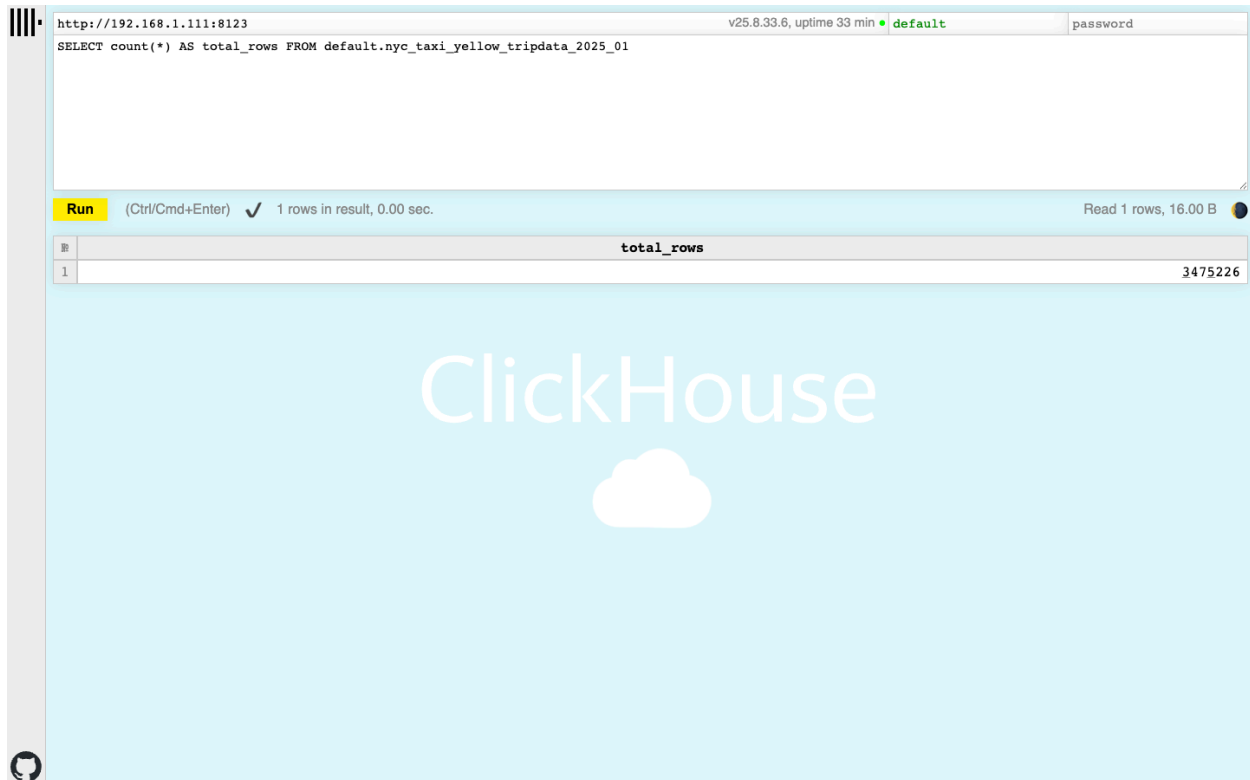
Detalle de los archivos `.metadata.json`, `.avro` y listas de manifiestos que genera Iceberg. Esta metadata es lo que convierte un conjunto de Parquet sueltos en una tabla con snapshots e historial.

Figura 5 — Catálogo Nessie con el namespace



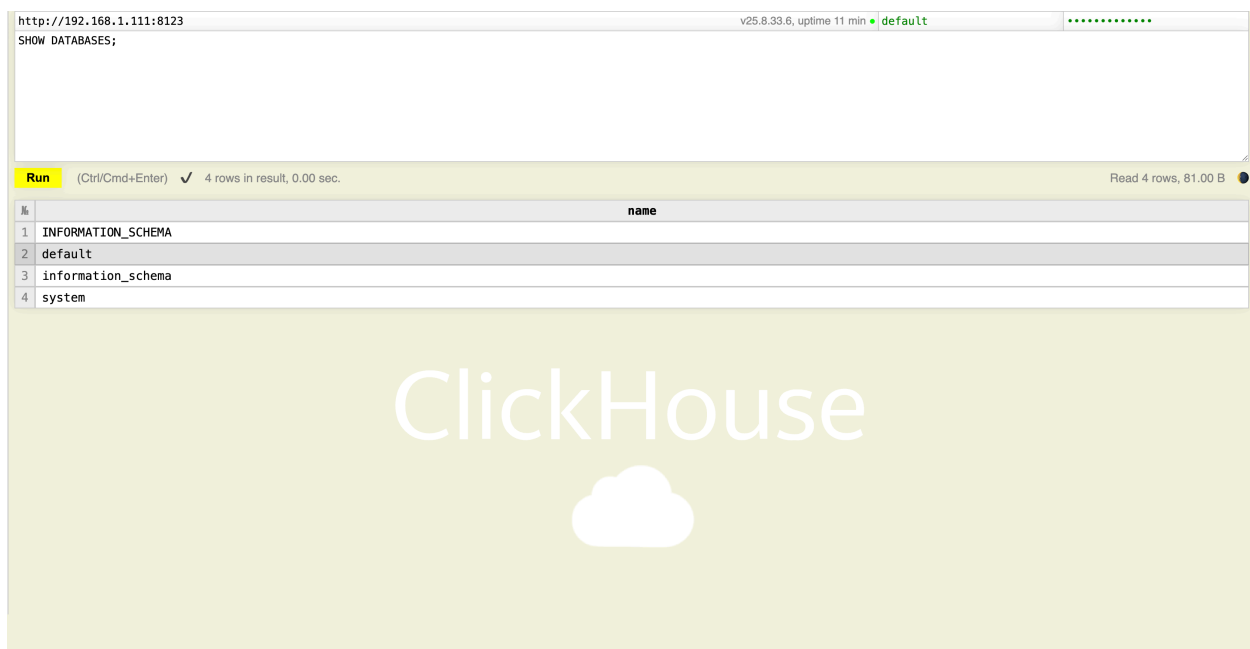
Namespace `nyc_taxi` y tabla `yellow_tripdata_2025_01` registrados en Nessie, con su *Metadata Location* apuntando al bucket Iceberg en MinIO.

Figura 6 — Conteo de registros en ClickHouse



Resultado de `SELECT count(*)` sobre la tabla final: **3.475.226 registros**, exactamente el valor exigido por el enunciado.

Figura 7 — Bases de datos y tablas de metadatos en ClickHouse



Salida de `SHOW DATABASES` : junto a las bases del sistema (`system` , `information_schema` , `INFORMATION_SCHEMA`) aparece `default` , que es el esquema donde `dlt` escribe los datos.



Salida de `SHOW TABLES FROM default` : la tabla de datos junto a las tablas de metadatos que genera `dlt` (`_dlt_loads` , `_dlt_pipeline_state` , `_dlt_version` y la tabla centinela).

Figura 8 — Archivo cargado en Azure ADLS

```
[5]: import tomllib

with open("/home/jovyan/.dlt/secrets.toml", "rb") as f:
    s = tomllib.load(f)

creds = s["minio_to_azure"]["destination"]["filesystem"]["credentials"]
account = creds["azure_storage_account_name"]
key = creds["azure_storage_account_key"]
bucket_url = s["minio_to_azure"]["destination"]["filesystem"]["bucket_url"]

print("Cuenta:", account)
print("Key cargada:", len(key), "caracteres (no se muestra)")
print("Destino configurado:", bucket_url)

Cuenta: fhbd
Key cargada: 88 caracteres (no se muestra)
Destino configurado: abfss://clase-4-dlt@fhbd.dfs.core.windows.net/GRUPO_2

[6]: import adlfs

fs = adlfs.AzureBlobFileSystem(account_name=account, account_key=key)
print("Containers en la cuenta:", fs.ls("/"))

Containers en la cuenta: ['clase-4-dlt']

[7]: container = "clase-4-dlt"
print("Contenido de la raíz del container", container, ":")
print(fs.ls(f"{container}/"))

Contenido de la raíz del container clase-4-dlt :
['clase-4-dlt/GRUPO_1', 'clase-4-dlt/GRUPO_2', 'clase-4-dlt/GRUPO_3', 'clase-4-dlt/GRUPO_4', 'clase-4-dlt/GRUPO_5', 'clase-4-dlt/GRUPO_6',
'clase-4-dlt/GRUPO_7', 'clase-4-dlt/GRUPO_SAMUEL', 'clase-4-dlt/GRUPO_Samuel-Felipe', 'clase-4-dlt/GRUPO_X', 'clase-4-dlt/taxis_parquet']

[8]: path = "clase-4-dlt/GRUPO_2"
try:
```

Listado del contenedor `clase-4-dlt` mediante la librería `adlfs`, mostrando la carpeta `GRUPO_2/nyc_taxi/` con los data files y la metadata. La verificación es programática, sin depender del portal de Azure.

Figura 9 — Verificación integral: 12/12 checks

Verificación integral del flujo (equivalente a scripts/verificar.py)

Ejecuta los 12 checks: MinIO, Iceberg, Nessie, ClickHouse y Azure. Solo lectura: no escribe datos en ninguna capa.

```
[1]: import sys
sys.path.insert(0, "/home/jovyan/scripts")

import runpy

# Ejecuta scripts/verificar.py tal cual (misma lógica, cero duplicación).
# verificar.py termina con sys.exit(0) al pasar todos los checks;
# se captura SystemExit para que el notebook no muestre una celda con error.
try:
    runpy.run_path("/home/jovyan/scripts/verificar.py", run_name="__main__")
except SystemExit as e:
    print(f"verificar.py terminó con exit code {e.code}")
```

```
=====
PROYECTO CORTE 1 | evidence report
=====

MinIO | Data Lake
=====
Buckets: ['nyc-taxi-iceberg', 'nyc-taxi-raw']

[OK ] bucket 'nyc-taxi-raw' exists
[OK ] bucket 'nyc-taxi-iceberg' exists
[OK ] RAW bucket has Parquet files - 1 file(s)
      nyc-taxi-raw/raw/yellow_tripdata_1788401857.0521398.08e971fb2c.parquet
[OK ] Iceberg bucket has data files - 42 file(s)
[OK ] Iceberg bucket has metadata files - 129 file(s)

=====
Nessie | Catalog
=====
Namespaces: ['nyc_taxi']

[OK ] namespace 'nyc_taxi' exists
[OK ] table 'nyc_taxi.yellow_tripdata_2025_01' registered
```

Ejecución de `verificar.py` recorriendo todo el stack en una sola corrida: buckets, archivos RAW, tabla Iceberg, catálogo Nessie, ClickHouse y Azure.

8. Incidentes y soluciones

Incidente	Causa	Solución
Nessie salía con <code>Permission denied</code> en <code>/nessie/data/LOG</code>	Docker crea el volumen nombrado como <code>root:root</code> cuando la ruta no existe en la imagen; Nessie corre como usuario <code>nessie</code> (uid 10000) y no puede escribir	Servicio <code>volume-init</code> en el <code>docker-compose.yml</code> : cede la propiedad de los volúmenes antes de que arranquen los servicios que dependen de ellos
<code>PermissionError</code> en <code>/home/jovyan/state/pipelines</code>	Mismo caso en el volumen <code>jupyter-state</code> ; el contenedor corre como <code>jovyan</code> (uid 1000)	Cubierto por el mismo servicio <code>volume-init</code>
dlt rechazaba <code>secrets.toml</code> (<code>Empty key at line 1 col 0</code>)	El archivo se generó en UTF-8 con BOM	Regenerar el archivo en UTF-8 sin BOM

Incidente	Causa	Solución
	(PowerShell 5.1); el parser TOML no acepta BOM	
<code>docker compose up</code> fallaba con <code>logon session does not exist</code>	El CLI de Docker en sesión SSH no accede al credential store de la sesión interactiva de Windows	Ejecutar el <code>up</code> desde la sesión interactiva del usuario
Key de Azure con 115 caracteres en <code>secrets.toml</code>	Se concatenó texto sobrante a la key (una key válida de Azure mide 88 caracteres)	Regenerar <code>secrets.toml</code> desde la plantilla con la key del enunciado, verificando longitud y hash
Azure respondía <code>AccountIsDisabled</code>	La cuenta de almacenamiento <code>fhbd</code> estaba deshabilitada del lado del servicio	El profesor re-habilitó la cuenta; el pipeline 03 completó la carga

9. Reproducción desde cero

La carpeta ya incluye `dlt/secrets.toml` y `.env` configurados.

```
docker compose down -v
docker compose up -d --build
docker ps
```

`down -v` borra los volúmenes: es necesario si el stack ya se levantó antes, porque un volumen creado sin el servicio `volume-init` conserva permisos de root. La primera build tarda unos minutos. `docker ps` debe mostrar 4 contenedores `Up`; `minio-init` y `volume-init` aparecen en `Exited (0)` porque son tareas de inicialización.

Luego, en Jupyter (`http://localhost:8888` , token `cortel`), ejecutar los notebooks en orden 01 → 02 → 03 → 04. O los scripts equivalentes:

```
docker compose exec jupyter python scripts/01_http_to_minio.p
y
docker compose exec jupyter python scripts/02_parquet_to_iceb
```

```

erg.py
docker compose exec jupyter python scripts/03_minio_to_azure.
py
docker compose exec jupyter python scripts/04_minio_to_clickh
ouse.py
docker compose exec jupyter python scripts/verificar.py

```

Consolas del stack:

Consola	URL	Credenciales
MinIO	<code>http://localhost:9001</code>	<code>admin</code> / <code>password123</code>
Nessie	<code>http://localhost:19120/tree/main/nyc_taxi</code>	—
ClickHouse (Play)	<code>http://localhost:8123/play</code>	<code>default</code> / <code>clickhouse123</code>
Jupyter	<code>http://localhost:8888</code>	token <code>corte1</code>

Los pipelines son idempotentes: re-ejecutarlos no duplica datos.

10. Conclusiones

El flujo queda completo de punta a punta: el dato público se ingiere con `dlt`, se persiste como Parquet en MinIO (data lake), se promueve a tabla Apache Iceberg con catálogo Nessie —separando file format, table format y catálogo como componentes independientes— y se distribuye a los dos destinos finales: Azure ADLS (carpeta `GRUPO_2`) y ClickHouse (data warehouse).

La validación crítica se cumple con exactitud: **3.475.226 filas** en las tres capas de datos, y la verificación integral cierra en **12/12 checks**, incluida la confirmación programática de los archivos en Azure. El stack es reproducible con un solo `docker compose up -d --build` y los secretos quedan fuera del código y del repositorio público en todo momento.